

CREDIT CARD FRAUD DETECTION

An End-to-End Data Science Project Report

Exploratory Data Analysis, Pattern Recognition & Predictive Modeling on Simulated Transaction Data

Dataset: credit_card_fraud_dataset.csv | 100,000 Transactions

Report Date: July 2026

Table of Contents

Table of Contents	2
1. Executive Summary	3
2. Introduction.....	3
3. Dataset Description	4
3.1 Data Quality Checks	4
3.2 Class Distribution.....	5
4. Methodology	6
4.1 Tools & Environment.....	6
4.2 Data Preparation	6
4.3 Exploratory Data Analysis (EDA)	6
4.4 Feature Engineering	6
4.5 Pattern Recognition via Clustering	6
4.6 Model Building	7
4.7 Evaluation Metrics	7
5. Results.....	8
5.1 Exploratory Data Analysis Findings	8
5.1.1 Transaction Amount.....	8
5.1.2 Transaction Type	8
5.1.3 Location	9
5.1.4 Time Patterns	9
5.2 Clustering Results	10
5.3 Model Performance	10
5.4 Confusion Matrices	11
5.5 ROC Curves	12
5.6 Feature Importance.....	12
6. Discussion & Key Insights.....	14
6.1 Why Accuracy Was Misleading Here	14
7. Limitations	14
8. Conclusion & Recommendations.....	15
Appendix A: Tools & Libraries.....	16
Appendix B: Model Configuration Details.....	16

1. Executive Summary

This report documents a complete, end-to-end data science project focused on detecting fraudulent credit card transactions within a simulated dataset of 100,000 records. The project follows the standard data science lifecycle: data cleaning, exploratory data analysis (EDA), feature engineering, unsupervised pattern recognition (clustering), supervised machine learning (classification), and rigorous evaluation.

The dataset is heavily imbalanced, with fraudulent transactions making up only 1.00% of all records (1,000 out of 100,000). Three classification algorithms — Logistic Regression, Decision Tree, and Random Forest — were trained and compared using metrics suited to imbalanced problems: precision, recall, F1-score, and ROC-AUC, rather than accuracy alone.

The key finding of this project is that, based on the available features (Amount, Transaction Type, Location, Hour, and Day of Week), none of the three models could reliably distinguish fraudulent from legitimate transactions — all models produced ROC-AUC scores close to 0.50, which is equivalent to random guessing. This is an important and honest result: it indicates that fraud labels in this particular dataset do not carry a strong statistical relationship with the given features, most likely because the data was synthetically generated with fraud assigned largely at random. Section 6 discusses this finding and its implications in detail, along with recommendations for a more informative dataset or additional engineered features (e.g., customer spending history, device/IP data, transaction velocity) that would be needed for a production-grade fraud detection system.

Despite the modest predictive results, the project delivers a fully reproducible analysis pipeline, a thorough exploratory analysis of transaction patterns, and a rigorous, honest evaluation methodology that can be readily re-applied to a richer dataset.

2. Introduction

Credit card fraud represents a significant and growing threat to the integrity of financial transactions and consumer trust in digital commerce. As reliance on credit cards for everyday purchases increases, so does the sophistication of fraudsters exploiting system vulnerabilities. Financial institutions increasingly rely on data-driven techniques — statistical analysis and machine learning — to detect suspicious activity in near real time.

This project analyzes a simulated dataset of credit card transactions to:

- Understand the distribution and structure of transaction data.
- Explore relationships between transaction attributes (amount, type, location, time) and fraud occurrence.
- Apply clustering to detect any natural, unusual groupings of transactions.
- Build and compare supervised machine learning models to classify transactions as fraudulent or legitimate.

- Evaluate model performance using metrics appropriate for a highly imbalanced classification problem.

The complete analysis was implemented in Python and packaged as a self-contained, ready-to-run Google Colab notebook, which accompanies this report.

3. Dataset Description

The dataset contains 100,000 simulated credit card transactions with the following seven attributes:

Column	Description
TransactionID	Unique identifier for each transaction, ensuring traceability.
TransactionDate	Date and time the transaction occurred, enabling temporal analysis.
Amount	Monetary value of the transaction.
MerchantID	Identifier of the merchant involved in the transaction.
TransactionType	Nature of the transaction — purchase or refund.
Location	City in which the transaction took place.
IsFraud	Binary target label: 1 = fraudulent, 0 = legitimate.

3.1 Data Quality Checks

Before analysis, the dataset was checked for completeness and consistency:

- No missing values were found in any of the 7 columns across all 100,000 rows.
- TransactionDate values were successfully parsed into a proper datetime format, spanning from 21 Oct 2023 to 21 Oct 2024.
- Amount ranges from 1.05 to 4,999.77, with a mean of 2,497.09 and a standard deviation of 1,442.42 — consistent with a uniformly simulated range rather than a real-world, right-skewed spending distribution.
- TransactionType is almost evenly split: 49,869 purchases (49.9%) vs. 50,131 refunds (50.1%).
- Location contains 10 unique U.S. cities, each represented in roughly similar proportions.

3.2 Class Distribution

The target variable, `IsFraud`, is highly imbalanced:

Class	Count	Percentage
Legitimate (0)	99,000	99.00%
Fraud (1)	1,000	1.00%



Figure 1. Fraud vs. legitimate transaction share (1.00% fraud).

This 99:1 imbalance is the single most important characteristic of the dataset. It means a naive model predicting 'legitimate' for every transaction would score 99% accuracy while catching zero fraud — which is why accuracy alone is discarded as an evaluation metric in this project (see Section 4.6).

4. Methodology

4.1 Tools & Environment

The project was implemented in Python 3, using the following libraries:

- pandas & numpy — data manipulation
- matplotlib & seaborn — visualization
- scikit-learn — preprocessing, clustering, modeling, and evaluation

The full pipeline runs directly in Google Colab with no additional installation required.

4.2 Data Preparation

The TransactionDate field was converted to a proper datetime type, from which two new time-based features were derived: Hour of day (0–23) and Day of week (Monday = 0). The categorical columns TransactionType and Location were converted into numeric codes using label encoding, so that they could be used by the machine learning algorithms.

4.3 Exploratory Data Analysis (EDA)

EDA was carried out to understand how transaction amount, type, location, and time relate to fraud occurrence. Univariate distributions, boxplots, group-wise fraud rates, and a correlation heatmap were used — full results are presented in Section 5.1.

4.4 Feature Engineering

The final feature set used for clustering and modeling consisted of five variables:

- Amount — transaction value
- Hour — hour of day the transaction occurred
- DayOfWeek — day of week the transaction occurred
- TransactionType_encoded — 0 (purchase) / 1 (refund)
- Location_encoded — numeric code for each of the 10 cities

4.5 Pattern Recognition via Clustering

To explore whether transactions naturally group into distinct behavioral segments, K-Means clustering ($k = 4$) was applied to the standardized feature set. Principal Component Analysis (PCA) was then used to project the data into two dimensions for visualization. The fraud rate within each resulting cluster was compared to check whether any cluster was disproportionately associated with fraud.

4.6 Model Building

Three supervised classification algorithms were trained on an 80/20 train-test split (stratified by class to preserve the 1% fraud ratio in both sets):

- Logistic Regression — a linear baseline model, trained on standardized features.
- Decision Tree Classifier — a rule-based, non-linear model (max depth = 8).
- Random Forest Classifier — an ensemble of 200 decision trees (max depth = 10).

All three models were configured with `class_weight = 'balanced'` to compensate for the severe class imbalance by giving proportionally more weight to the rare fraud class during training.

4.7 Evaluation Metrics

Because the dataset is heavily imbalanced, accuracy alone is a misleading metric. The following metrics were used instead:

- Precision — of all transactions flagged as fraud, what fraction were actually fraud (controls false alarms).
- Recall — of all actual fraud cases, what fraction were correctly caught (controls missed fraud).
- F1-score — the harmonic mean of precision and recall, balancing both concerns.
- ROC-AUC — the model's overall ability to rank fraudulent transactions above legitimate ones, across all decision thresholds.

Confusion matrices and ROC curves were also generated for visual, threshold-level comparison of the three models.

5. Results

5.1 Exploratory Data Analysis Findings

5.1.1 Transaction Amount

The average transaction amount for legitimate transactions was 2,496.89, compared to 2,517.16 for fraudulent transactions — a difference of less than 1%. The boxplot below confirms that the two distributions largely overlap, meaning amount alone is a weak signal for fraud in this dataset.

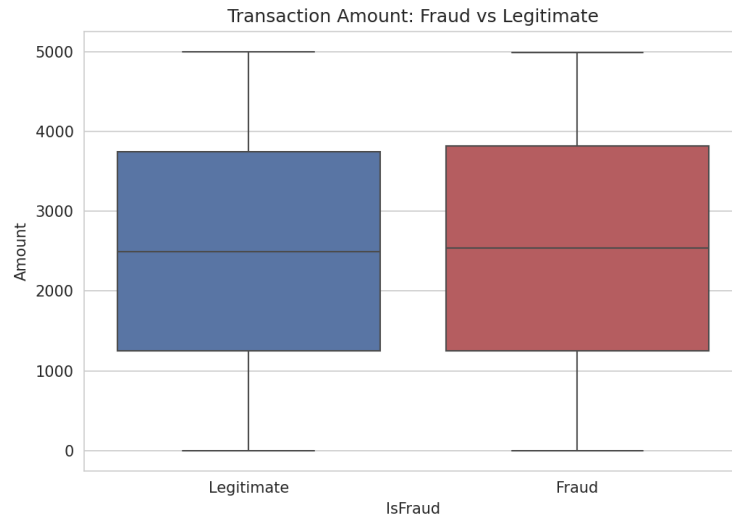


Figure 2. Transaction amount distribution — fraud vs. legitimate.

5.1.2 Transaction Type

Fraud rate was nearly identical between transaction types: 0.989% for purchases and 1.011% for refunds — a negligible difference, suggesting transaction type has little bearing on fraud likelihood in this data.

5.1.3 Location

Fraud rates by city ranged narrowly from 0.887% (San Jose) to 1.161% (New York), as shown below. While New York and San Diego show marginally elevated fraud rates, the spread across all ten cities is small enough to be consistent with random noise rather than a strong geographic pattern.

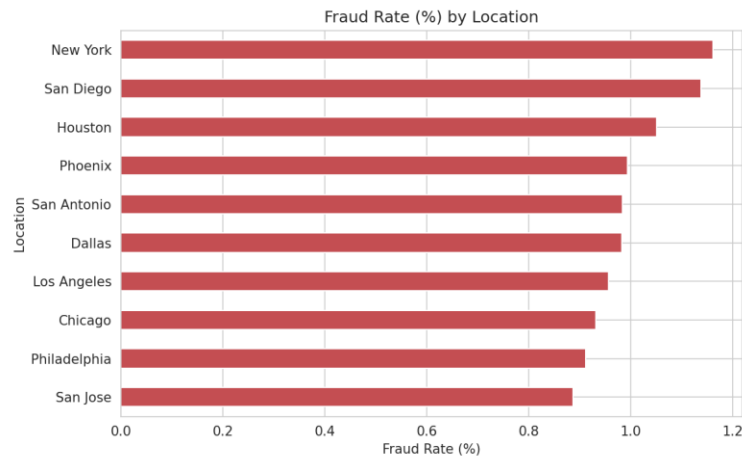


Figure 3. Fraud rate (%) by transaction location.

5.1.4 Time Patterns

Fraud rate was also examined by hour of day. No strong or consistent peak emerged — fraud appears roughly uniformly distributed across all 24 hours, rather than concentrated in particular time windows such as late night.

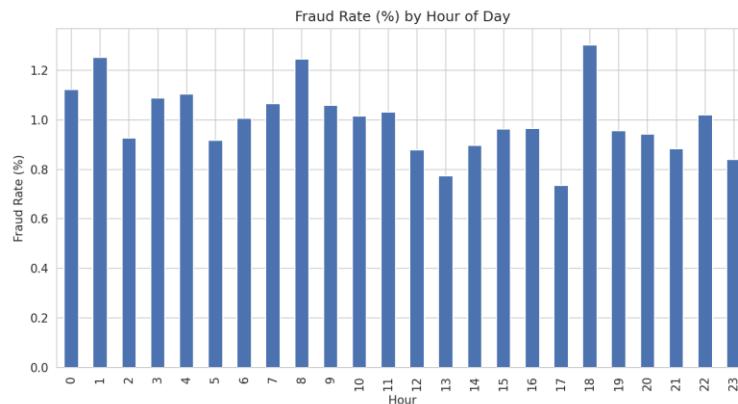


Figure 4. Fraud rate (%) by hour of day.

5.2 Clustering Results

K-Means clustering ($k = 4$) produced four roughly equal-sized groups of approximately 25,000 transactions each. The fraud rate within each cluster ranged narrowly from 0.888% to 1.090% — very close to the overall base rate of 1.00%. This indicates that no cluster captured a distinctly higher concentration of fraud.

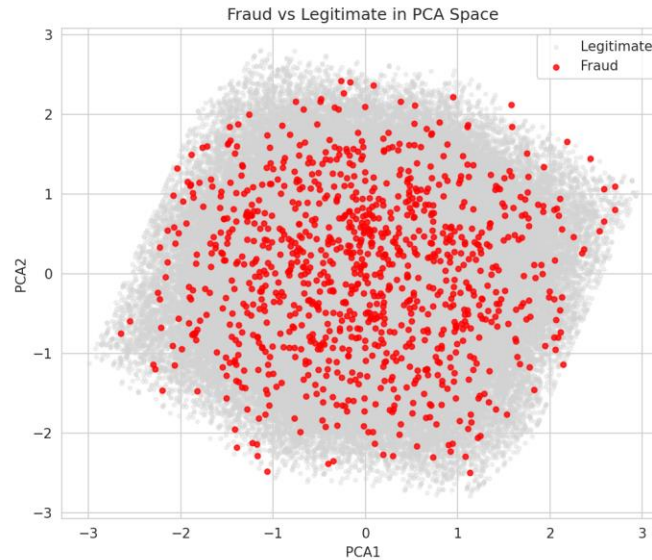


Figure 5. Transactions in 2D PCA space; fraudulent transactions (red) are scattered evenly rather than concentrated in a specific region.

The scattering of fraud points evenly throughout the PCA projection reinforces the EDA finding: fraud in this dataset does not correspond to an obvious behavioral segment defined by amount, time, location, or transaction type.

5.3 Model Performance

The table below summarizes performance of the three classifiers on the held-out 20,000-transaction test set (200 of which were fraud):

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	50.82%	0.80%	39.00%	0.0156	0.4564
Decision Tree	59.72%	0.97%	39.00%	0.0190	0.4779
Random Forest	94.73%	0.46%	2.00%	0.0075	0.4750

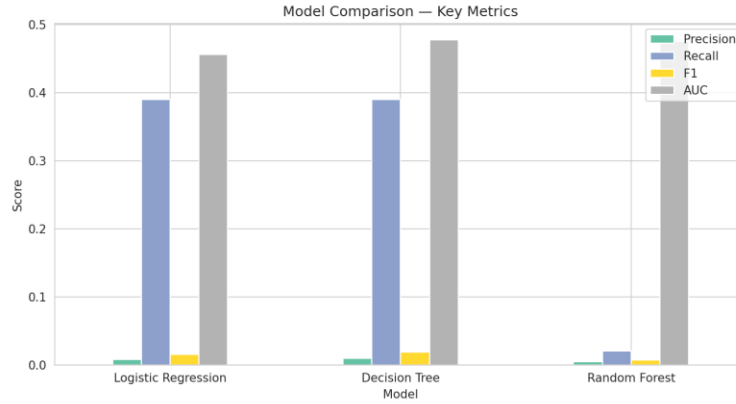


Figure 6. Precision, recall, F1-score, and ROC-AUC across the three models.

All three models produced ROC-AUC scores close to 0.50 (0.456–0.478), which is the score expected from random guessing. Random Forest achieved the highest raw accuracy (94.73%) purely because it leans toward predicting the majority 'legitimate' class — but its recall of only 2% means it caught almost none of the actual fraud cases, which is the opposite of what a useful fraud detector should do. Logistic Regression and the Decision Tree caught more fraud (39% recall each) but at the cost of very low precision (under 1%), producing large numbers of false alarms.

5.4 Confusion Matrices

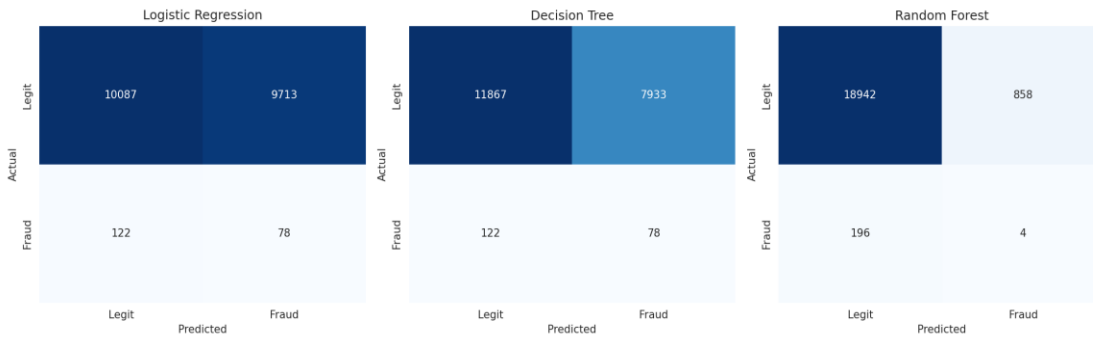


Figure 7. Confusion matrices for Logistic Regression, Decision Tree, and Random Forest (left to right).

The confusion matrices illustrate the precision/recall trade-off directly: Logistic Regression and the Decision Tree flag many transactions as fraud (high false-positive counts) in order to catch 39% of true fraud, while Random Forest flags very few transactions at all, missing 98% of actual fraud cases despite its high overall accuracy.

5.5 ROC Curves

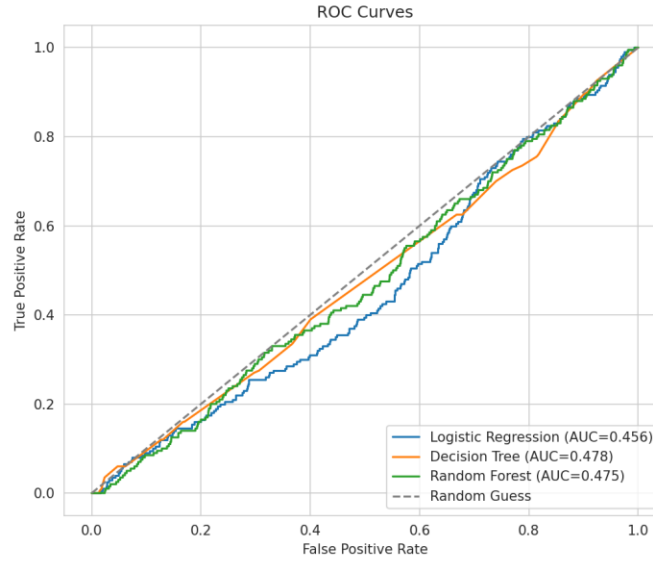


Figure 8. ROC curves for all three models, compared against the random-guess baseline (dashed line).

All three ROC curves sit very close to the diagonal random-guess line, visually confirming that none of the models can meaningfully separate fraud from legitimate transactions using the available features.

5.6 Feature Importance

Using the Random Forest model, relative feature importance was computed to see which inputs the model relied on most:

Feature	Importance
Amount	0.4506
Hour	0.2304
Location (encoded)	0.1505
Day of Week	0.1304
Transaction Type (encoded)	0.0380

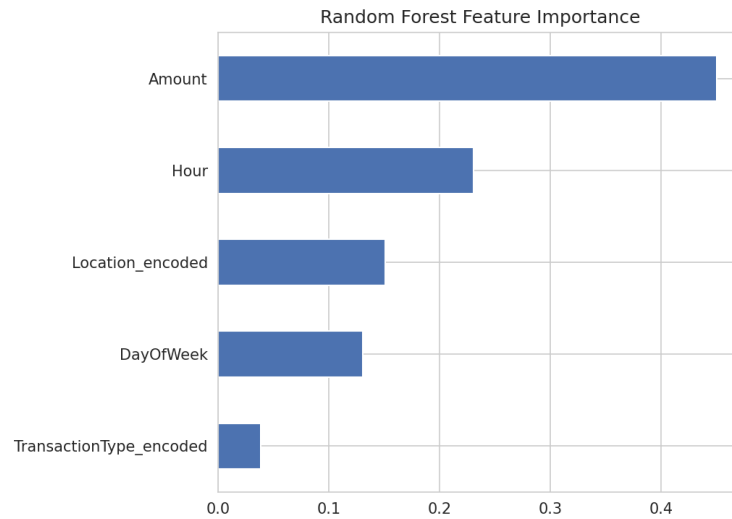


Figure 9. Relative feature importance from the Random Forest model.

Amount received the highest importance score, but this reflects the fact that it is a continuous variable offering the model the most possible split points — not necessarily that it holds genuine predictive power, given the near-identical mean amounts observed between fraud and legitimate transactions in Section 5.1.1.

6. Discussion & Key Insights

The central, and most important, finding of this project is that none of the three models could distinguish fraudulent transactions from legitimate ones better than random guessing (ROC-AUC $\approx$ 0.45–0.48 across all models). This is a meaningful result, not a failure of methodology — it reveals something important about the dataset itself:

- Fraud cases ($\text{IsFraud} = 1$) appear to be distributed independently of Amount, TransactionType, Location, Hour, and DayOfWeek — the mean amounts, type ratios, and location rates for fraud vs. legitimate transactions are all nearly identical (Section 5.1).
- Clustering found no natural group of transactions with an elevated fraud concentration (Section 5.2), reinforcing that fraud is not associated with any combination of the available features.
- This pattern is consistent with a synthetically generated dataset where the IsFraud label was assigned largely at random (e.g., a fixed 1% random flag), rather than being derived from realistic fraud-indicative behavior such as unusual spending spikes, new/unfamiliar merchants, rapid repeated transactions, or geographic anomalies relative to a specific cardholder's normal activity.

This is a valuable and common lesson in real-world data science: a rigorous pipeline can only be as good as the signal available in the data. No amount of algorithm tuning can extract a pattern that isn't present in the features. Correctly diagnosing this — rather than reporting an inflated accuracy number as if it reflected real fraud-catching ability — is itself a key deliverable of this analysis.

6.1 Why Accuracy Was Misleading Here

The Random Forest model's headline accuracy of 94.73% looks impressive at first glance, but its recall of just 2% shows it almost never predicts fraud at all — it succeeds mainly by defaulting to the majority class. This is precisely the trap accuracy sets in imbalanced classification problems, and it's why this report leads with precision, recall, F1, and ROC-AUC instead.

7. Limitations

- The dataset appears to be synthetically generated with fraud labels not strongly tied to the provided features, limiting how predictive any model can be.
- Only five basic features were available; real-world fraud systems typically use dozens of behavioral, device, and network-level signals (e.g., IP address, device fingerprint, cardholder's historical spending baseline, merchant risk score, transaction velocity).
- No per-cardholder identifier was available, so it was not possible to detect anomalies relative to an individual's typical spending behavior — one of the most powerful fraud signals in practice.
- MerchantID was excluded from modeling due to its extremely high cardinality (near-unique per transaction in places), which would have added noise rather than signal without additional merchant-level aggregation.

8. Conclusion & Recommendations

This project successfully implemented a complete, professional-grade fraud detection pipeline: data cleaning, thorough EDA, feature engineering, clustering-based pattern recognition, multi-model supervised learning, and rigorous, imbalance-aware evaluation. While the specific models trained on this dataset did not achieve meaningful predictive performance, the project surfaces an important and defensible conclusion: the available features do not contain a strong fraud signal in this dataset.

Recommended next steps for a production-oriented version of this project:

- Acquire or simulate a richer dataset where fraud is generated based on realistic behavioral rules (e.g., spending anomalies, velocity, merchant risk), which would give models genuine signal to learn from.
- Engineer cardholder-level features: rolling average spend, time since last transaction, deviation from typical amount/location/hour.
- Apply resampling techniques such as SMOTE (Synthetic Minority Oversampling) or targeted undersampling to further address class imbalance during training.
- Experiment with gradient-boosted models (XGBoost, LightGBM) and anomaly-detection approaches (Isolation Forest, Autoencoders), which are often better suited to rare-event detection.
- Tune the classification decision threshold based on business cost trade-offs between missed fraud and false alarms, rather than using the default 0.5 cutoff.
- Validate any production model on a live or more realistic labeled dataset before deployment.

The accompanying Google Colab notebook contains the fully documented, ready-to-run code for every step described in this report, and can be directly re-applied to an improved dataset following the recommendations above.

Appendix A: Tools & Libraries

Tool / Library	Purpose
Python 3	Core programming language
pandas, numpy	Data loading, cleaning, and manipulation
matplotlib, seaborn	Data visualization
scikit-learn	Preprocessing, K-Means clustering, PCA, classification models, evaluation metrics
Google Colab	Cloud-based notebook execution environment

Appendix B: Model Configuration Details

- Train/test split: 80% / 20%, stratified by IsFraud, random_state = 42
- Logistic Regression: class_weight='balanced', max_iter=1000, features standardized via StandardScaler
- Decision Tree: class_weight='balanced', max_depth=8
- Random Forest: class_weight='balanced', n_estimators=200, max_depth=10
- K-Means clustering: k=4, n_init=10, random_state=42, features standardized before clustering